What Makes Neural Networks Easy to Fool? A Controlled Tour of the Candidate Root Causes

Ali Kayyam
akayyam@brainchip.com

Abstract

Networks are now much more robust to small, deliberate changes than they were a few years ago, yet there is no agreed answer to the basic question: why are ordinary networks fragile in the first place? Candidate explanations abound. Networks add up many tiny effects, rely on predictive but fragile features, prefer texture to shape, ignore directions the data never uses, lack data, or are led by training to the wrong solution. They are rarely compared on equal terms. This paper is an illustrative study, written for junior researchers and curious readers. It tests each explanation in a small, fully controlled world with one yardstick: the smallest change that flips the model's answer, compared with the best that any model could do. We sort the findings into *inevitable* vulnerability (no model can be both accurate and robust at the allowed change) and *avoidable* vulnerability (a better model exists, but training does not find it). In our toy worlds the dominant cause is inevitable: evidence spread over many inputs under a per-input budget. It also accounts for shape versus texture, where only how widely a cue is spread matters, not its frequency. On top of it sit smaller avoidable effects: ignoring irrelevant inputs (up to 2×), the optimiser's bias towards the wrong kind of solution, a convolutional network that reads fine patterns badly, and sensitivity off the data manifold. Adversarial training closes the whole avoidable gap in the toy world, at a cost in clean accuracy. More data does not change the vulnerability of standard models, but strongly affects adversarially trained ones. Checks on MNIST, Fashion-MNIST and CIFAR-10, with small and large networks, confirm most of this and qualify the rest. The paper is an aid to intuition and a map of causes, not a claim about large networks in general.

1 The puzzle and a way to organise the answer

A faint pattern of noise, invisible to a person, makes an accurate image classifier change its answer. Adversarial training and other defences now make models much more robust, but we still lack a clear account of *why* ordinary models are fragile in the first place. The literature offers many candidate causes [1, 2, 3, 4, 5]; each is usually shown on its own, on a different dataset, with a different measure.

We do two things. First, we compare candidate causes in the same small, fully controlled worlds with the same measure. Second, we sort what we find using one distinction.

- **Inevitable (data-limited).** Given the data and the allowed change ϵ, no classifier can be both accurate and robust. The classes overlap within ϵ, or the evidence is spread so thinly that robustness costs accuracy.
- **Avoidable (learning-limited).** A classifier that is accurate *and* robust exists, but the training objective, the optimiser, the architecture or the amount of data does not find it.

Figure 1 shows the two cases in a picture. Every hypothesis below is a claim about which part it explains. A cause that is *necessary* must appear whenever vulnerability does; a cause that is *sufficient* produces vulnerability by itself; a cause may also only *contribute*. Our scorecard (Section 5) records which, with a plain statement of how strong the evidence is.

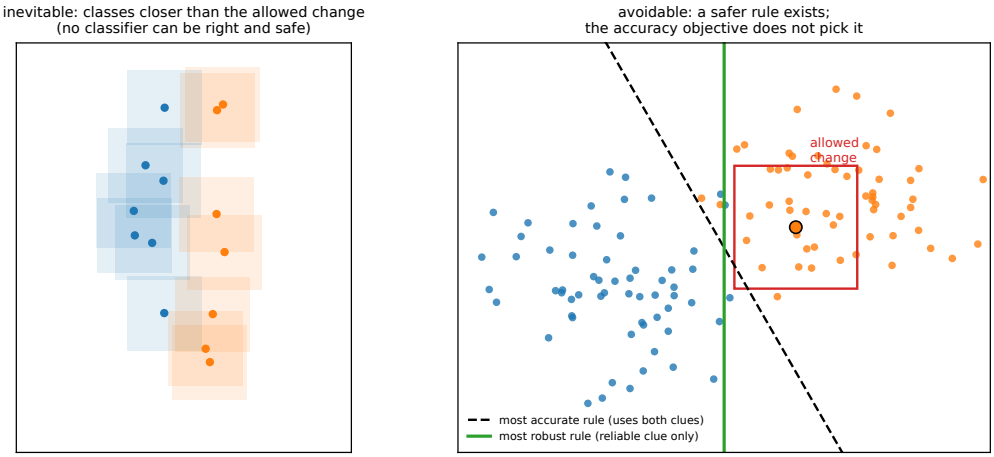


Figure 1: Two kinds of vulnerability. Left: when the allowed change (squares around each point) is larger than the gap between the classes, no classifier can be both right and safe: inevitable. Right: two clues, one reliable and one weak. The most accurate rule (dashed) uses both and tilts; the most robust rule (green) uses the reliable clue only. A change within the red square flips the accurate rule's answer for the marked point but not the robust rule's: avoidable, but only by changing what training asks for.

The yardstick. For every model and every correctly classified test point we compute the *flip size*: the smallest change that flips the answer. We measure it two ways: the largest change to any single input (per-input, ℓ_∞) and the length of the whole change (total, ℓ_2). Their ratio tells how widely the model spreads its attention. We also report *accuracy under attack*: the share of test points still classified correctly after the worst change of a given size. Where the data are generated by us, we compute the same quantities exactly for the best possible rule.

How to read the paper. Each subsection of Section 3 asks one question, runs one small experiment, and ends with a boxed verdict. Section 4 repeats the checks on real images, and Section 5 collects everything in a scorecard. Settings are in Appendix A; unless stated, numbers are means over 10 runs.

2 The toy world

Every example is six numbers and a label 0 or 1 with equal chance. The first number is a *reliable clue* (its class averages are 1.0 apart; looking at it alone gives 95% accuracy). The other five are *weak clues* (class averages 0.4 apart; each alone gives 79%). The noise is bell-shaped and the values are not clipped, so the best possible rule is a weighted sum of the six numbers, answered by its sign. It reaches 99.3%. Five weak clues together reach 96.3%, better than the reliable clue alone. A small network (one hidden layer of 8 units) trained on this task reaches the same accuracy.

For a weighted sum with weights w_i and total $\sum_i w_i x_i$, an attacker who may change every input by at most ϵ moves the total by $\epsilon \sum_i |w_i| = \epsilon \|w\|_1$. The answer flips when this exceeds the answer's confidence, so

$$\text{per-input flip size} = \frac{\text{confidence}}{\|w\|_1}, \qquad \text{total flip size} = \frac{\text{confidence}}{\|w\|_2}.$$

This one formula organises most of what follows. It is exact for the best rule; for trained networks we find the flip size by search.

3 The tour

3.1 H1: evidence added up over many inputs

Claim (Goodfellow et al. [2]; see also Simon-Gabriel et al. [12]): a small change to every input, all in the unhelpful direction, adds up. **Experiment:** keep the total evidence fixed and split it equally over $k = 1, 2, \ldots, 64$ inputs, so accuracy stays at 95% (Figure 2).

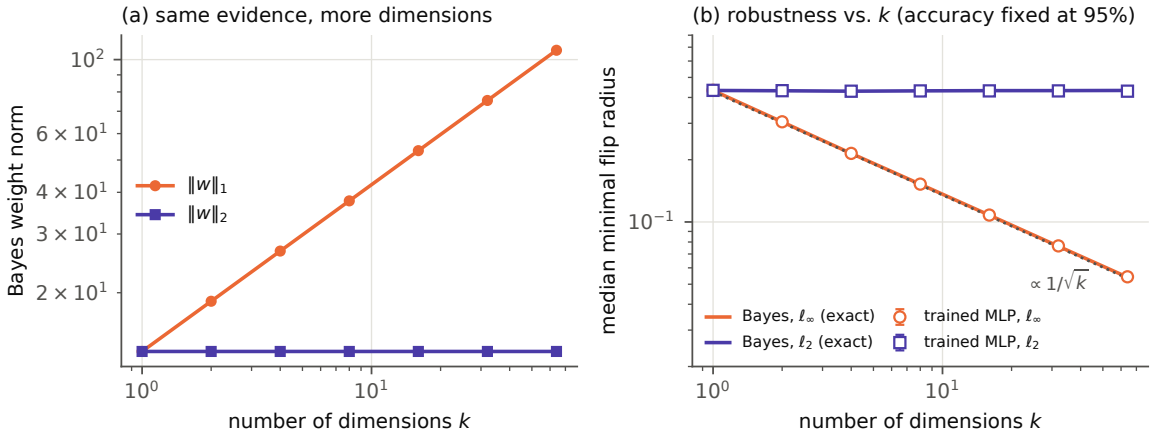


Figure 2: Same evidence spread over k inputs. Per-input flip size (steep lines) falls as $1/\sqrt{k}$; total flip size (flat lines) does not move. Exact best rule (lines) and trained networks (markers) coincide.

Figure 3 shows why in two dimensions. The straight-line distance to the boundary (the circle) is the same in both panels, but a diagonal boundary is reached by a smaller square of per-input changes than an axis-aligned one, because the square's corner moves both inputs at once.

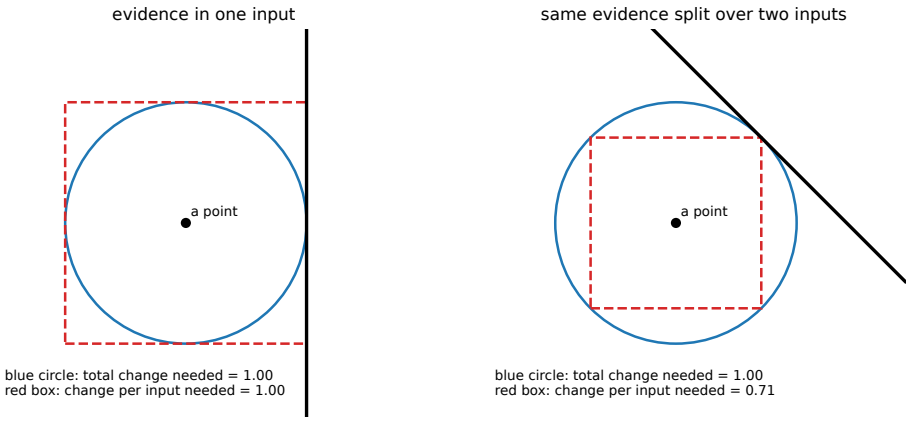


Figure 3: Two inputs, one point. Blue circle: total change needed to reach the decision boundary (black). Red dashed square: per-input change needed. Splitting the evidence over two inputs leaves the circle unchanged and shrinks the square.

The per-input flip size falls from 0.43 at $k = 1$ to 0.054 at $k = 64$ (a factor of eight), with the same accuracy and the same total flip size (0.43). The trained network lies on the exact curve.

Verdict. Necessary and sufficient, in this world, for fragility to small per-input changes. It is *inevitable*: the best rule has it too. It disappears if the attacker is limited by total size (ℓ_2) instead, so it is a statement about the pairing of the data with the allowed change.

3.2 H2: small clues, not unreliable ones

Claim (Ilyas et al. [4]): networks rely on features that are predictive but fragile. **Experiment:** one clue; vary its size (gap between class averages) and its reliability independently (Figure 4).

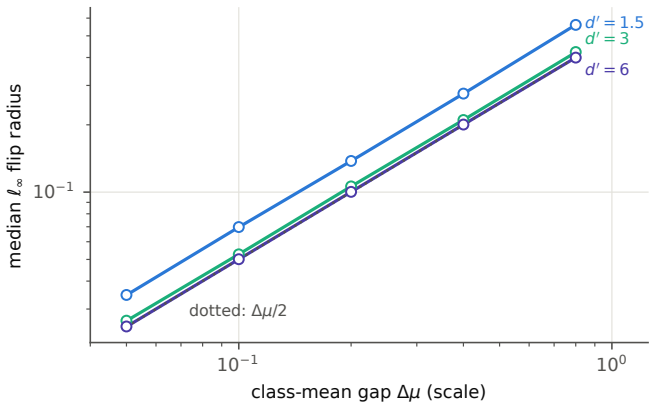


Figure 4: Flip size of a single clue against its size, at three reliabilities. The lines nearly coincide: size matters, reliability hardly does.

Flip size is proportional to the clue's size. Raising reliability from 77% to 99.9% changes it by only about 1.4×, and in the wrong direction for the usual intuition: a more reliable clue is slightly *easier* to flip. A clue that is 99.9% reliable but tiny (gap 0.05) flips at 0.025.

Verdict. "Non-robust" means small relative to the allowed change, not unreliable. Inevitable (a property of the data). The next test asks whether adversarial changes are really made of such features.

Transfer test: are adversarial changes made of real features? Ilyas et al. [4] showed that a network trained only on adversarially changed images, with labels given by the *target* of the change, can still classify clean images. We repeat this in miniature. For each training point we craft a targeted change of size ϵ towards a randomly chosen target label and keep the changed input with that target label. The labels are random with respect to the true class, so a fresh network can do well on clean data only if the changes themselves carry features (Figure 5).

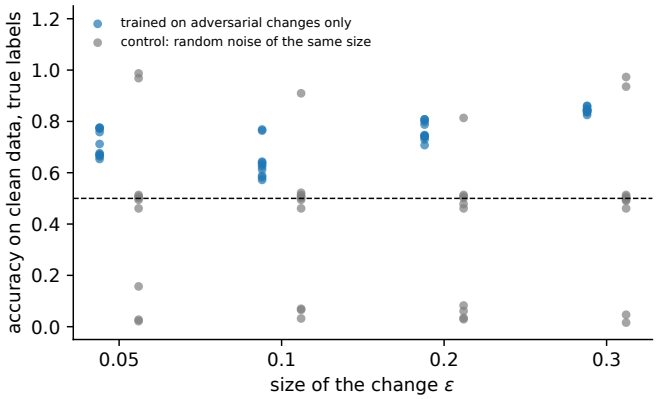


Figure 5: Accuracy on clean data (true labels) of a fresh network trained only on adversarially changed inputs with random target labels (blue, 10 runs), against a control trained on random-sign noise of the same size (gray).

The fresh network is above chance in every one of the 10 runs at every ϵ (mean 0.71, 0.64, 0.76, 0.84 for $\epsilon = 0.05, 0.1, 0.2, 0.3$). The control scatters between 0.02 and 0.99: an almost untrained network lines up with the reliable clue by chance, positively or negatively, and shows no consistent signal.

Verdict. In this world an adversarial change is built from the data's own predictive directions. This supports the "features, not bugs" reading in its simplest form. It is unsurprising for a weighted sum, which is the point: very little needs to be assumed.

3.3 H4: sparsity

Sparsity can mean three different things. We test each.

(a) Sparse versus dense signal. Put the same total evidence in s of 64 inputs; the other $64 - s$ inputs are pure noise (Figure 6).

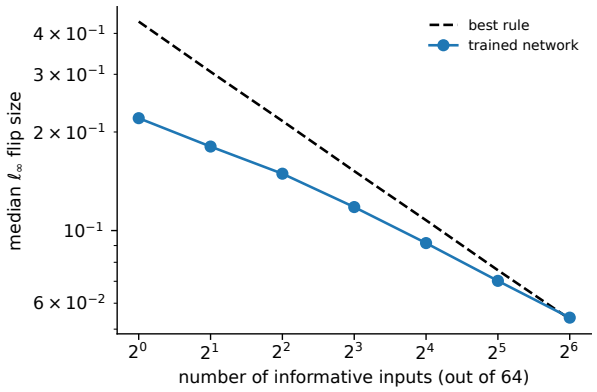


Figure 6: Median per-input flip size against the number of informative inputs (out of 64), same total evidence. Dashed: best rule. Blue: trained network.

The best rule's flip size falls as $1/\sqrt{s}$ (from 0.43 at $s = 1$ to 0.054 at $s = 64$): a dense signal is easier to fool. But the trained network is about *twice as fragile* as the best rule when the signal is sparse (0.22 against 0.43 at $s = 1$) and matches it when the signal is dense. The network does not fully ignore the irrelevant inputs.

(b) Sparse weights. In the six-clue task of Section 2, we measure the share of points flipped by an attack ($\epsilon = 0.3$) that may change only the five weak clues. Penalties that switch whole inputs off (first-layer L_1, group Lasso, elastic net) cut it from 0.260 to 0.02–0.05 at an accuracy cost of 1.5–3 points. Ordinary weight shrinkage (L_2) does not: it helps a little at no cost (0.26 $\rightarrow$ 0.21), then collapses to chance at strength 0.4.

(c) Sparse activity. Penalising hidden activity lowers activity (0.42 $\rightarrow$ 0.07) but leaves the weight pattern (reliable-to-weak weight ratio 1.97 $\rightarrow$ 1.81) and the weak-clue attack (0.26) essentially unchanged, and it fails all at once: each run either stays accurate or becomes a dead network (0 of 10 runs at strength 2, all 10 at strength 8).

Verdict. Sparsity of the *signal* is not a cause of fragility; *density* is (it is H1 again). What the model does with irrelevant inputs is an *avoidable*, learning-limited contribution of up to 2× here. Sparsity of the *weights* removes fragility at a price (accuracy); sparsity of the *activity* does not help.

3.4 H8: random noise versus the worst case

Claim (Ford et al. [7]): adversarial examples are the far tail of ordinary sensitivity to noise. **Experiment:** for the best rule, compare the size of random $\pm$ noise needed to flip a quarter of the points with the size of the worst-case change (Figure 7).

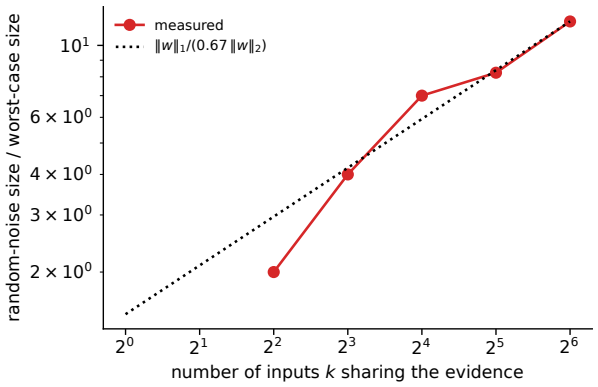


Figure 7: How many times larger random noise must be than the worst case to flip the answer, against the number of inputs k sharing the evidence. Dotted: $\|w\|_1/(0.67\|w\|_2)$.

The gap grows like $\sqrt{k}$ (about 2 at $k = 4$, 12 at $k = 64$) and follows $\|w\|_1/\|w\|_2$.

Verdict. Not a separate cause but the same one seen from another side: the more inputs share the evidence, the more the worst case beats random noise. No training is involved.

3.5 H10: the inevitable ceiling and the avoidable gap

Here we ask directly how much vulnerability is inevitable. For a linear rule with weights w the accuracy under the worst change of size ϵ is exactly $\Phi\big((\frac{1}{2}\sum_i w_i\Delta\mu_i - \epsilon\|w\|_1)/\sqrt{\sum_i w_i^2\sigma_i^2}\big)$ ($\Delta\mu_i$ the class gap, σ_i the noise of input i). We maximise it over w at each ϵ to get the best accuracy under attack that any linear rule can reach (Figure 8).

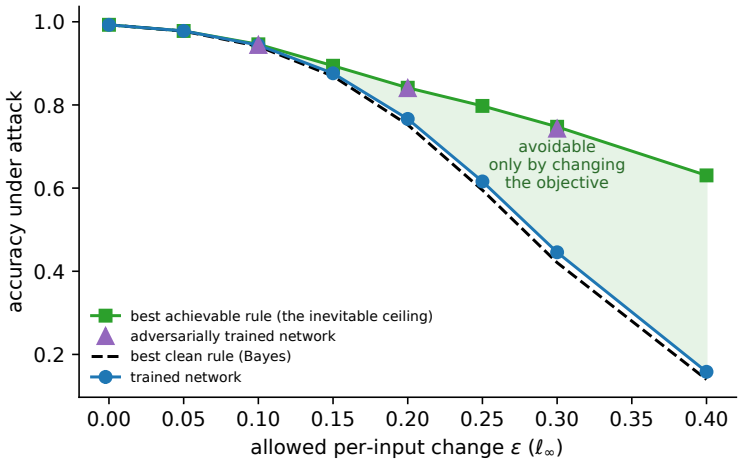


Figure 8: Accuracy under attack against the allowed per-input change. Green: the best achievable rule (the inevitable ceiling). Blue: a trained network. Purple triangles: the same network adversarially trained. Dashed: the best clean rule. The shaded gap is avoidable, but only by changing the training objective.

At $\epsilon = 0.3$ the ceiling is 0.75 (reached by using the reliable clue only), while the best clean rule gets 0.42 and the trained network 0.45. The ceiling itself falls as ϵ grows (0.63 at $\epsilon = 0.4$): part of the loss is inevitable. The gap between ceiling and network, about 0.3 at $\epsilon = 0.3$, is avoidable, yet *standard training will not close it*, because the standard objective rewards the clean-optimal rule. For small ϵ (≤ 0.1) the two coincide.

Does adversarial training close the gap? We trained the same network by adversarial training (a 5-step attack on every batch). At $\epsilon = 0.3$ it reaches 0.743 ± 0.011 accuracy under attack, against a ceiling of 0.748 and 0.44 for standard training; at $\epsilon = 0.2$ it reaches 0.840 against a ceiling of 0.841 (standard: 0.76); at $\epsilon = 0.1$ there is no gap to close. The price is clean accuracy: 0.954 and 0.960 against 0.993 (Figure 8, triangles).

Verdict. Both parts are real. The ceiling is inevitable. The gap is avoidable by changing the objective, and adversarial training closes it completely here, at the cost of the clean accuracy the weak clues carried: the accuracy/robustness trade-off of Tsipras et al. [3] in numbers. The ceiling is for linear rules only.

3.6 H7: which solution the optimiser reaches

Gradient descent on separable data is known to head for the solution with the largest margin measured in ℓ_2, not in ℓ_∞ (Soudry et al. [11]). **Experiment:** 100 inputs, s of them informative, 60 training examples (so the data are separable). Compare the ℓ_2 max-margin classifier (we use an SVM as the stand-in for the gradient-descent limit) with the ℓ_∞ max-margin classifier (a linear programme) and the oracle best rule (Figure 9).

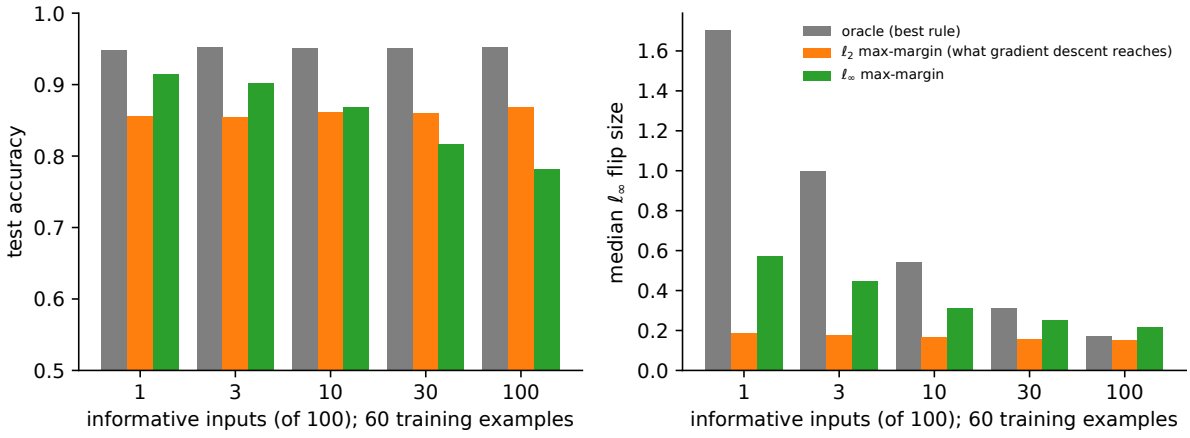


Figure 9: Accuracy (left) and per-input flip size (right) of three linear classifiers, against the number of informative inputs. The ℓ_∞ max-margin solution beats the ℓ_2 one when the signal is sparse.

Figure 10 shows the two solutions on a few points in two dimensions: the ℓ_2 solution tilts to use the irrelevant input, the ℓ_∞ solution does not.

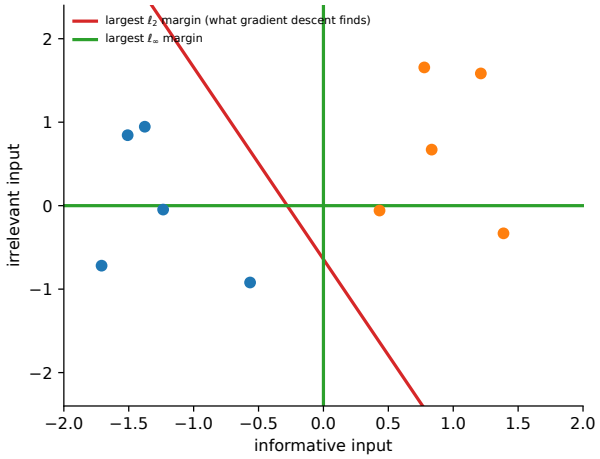


Figure 10: Ten training points; one input is informative, one irrelevant. Red: the largest-ℓ_2-margin separator, the solution gradient descent reaches. Green: the largest-ℓ_∞-margin separator, which ignores the irrelevant input.

With a sparse signal ($s = 1$) the ℓ_∞ solution is better on both axes (accuracy 0.92 against 0.86; flip size 0.57 against 0.19). With a dense signal the gap closes, and the ℓ_∞ solution then trades accuracy (0.78 against 0.87) for a larger flip size (0.22 against 0.15).

We checked the stand-in directly: 20 000 steps of gradient descent on the logistic loss from zero end in a direction with cosine similarity 0.98 to the SVM solution, with accuracy and flip size within about 0.015 of it (for $s = 1$: accuracy 0.869 against 0.855, flip size 0.199 against 0.186).

Verdict. The optimiser's bias towards ℓ_2 is an avoidable contribution when the truth is sparse and data are scarce. When the truth is dense, the gap is the trade-off again.

3.7 H6: more data

Claim (Schmidt et al. [8]): robust generalisation needs more data. **Experiment (standard training only):** train on $n = 30$ to 4000 examples, with the evidence spread over 16 inputs (Figure 11).

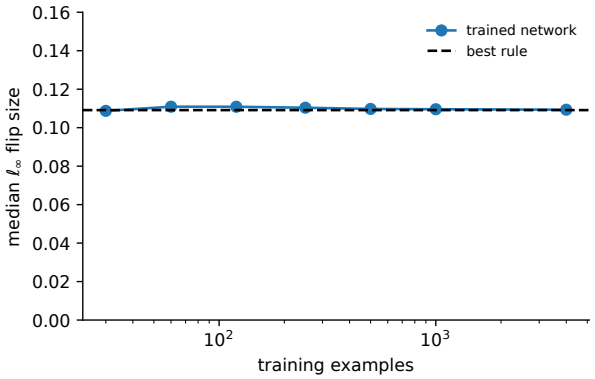


Figure 11: Per-input flip size against the number of training examples. The trained network matches the best rule at every n.

Accuracy rises from 0.905 to 0.953, but the flip size equals the best rule's (0.109–0.111) at every n.

With adversarial training. Schmidt et al.'s claim is about robust (adversarially trained) learning, so we repeat the experiment with it. In the toy world with 16, 64 and 256 inputs ($\epsilon = 0.05$, 0.03, 0.015; ceilings 0.81, 0.76, 0.76), adversarial and standard training give the same accuracy under attack at every n, both climbing to the ceiling (for 64 inputs: 0.62 at $n = 30$, 0.75 at $n = 1000$, 0.76 at $n = 16\,000$). The data needed to get within 2% of the ceiling grows roughly in proportion to the number of inputs: about 300 examples with 16 inputs, 1000 with 64 and 4000 with 256. On real data (Figure 12), standard networks have almost no accuracy under attack at any size (MNIST 0.01–0.05, CIFAR-10 0.03–0.06), while adversarially trained ones reach 0.25, 0.57, 0.69 (MNIST, $\epsilon = 0.1$, 100, 1000, 10 000 examples) and 0.22, 0.47, 0.57 (CIFAR-10, $\epsilon = 2/255$, 1000, 5000, 20 000 examples). Their flip sizes are 1.5–3× (MNIST) and 2–6× (CIFAR-10) those of standard networks, at a clean-accuracy cost of up to 6 points on CIFAR-10 and about 0–1 points on MNIST (where the cost is negative at 1000 examples).

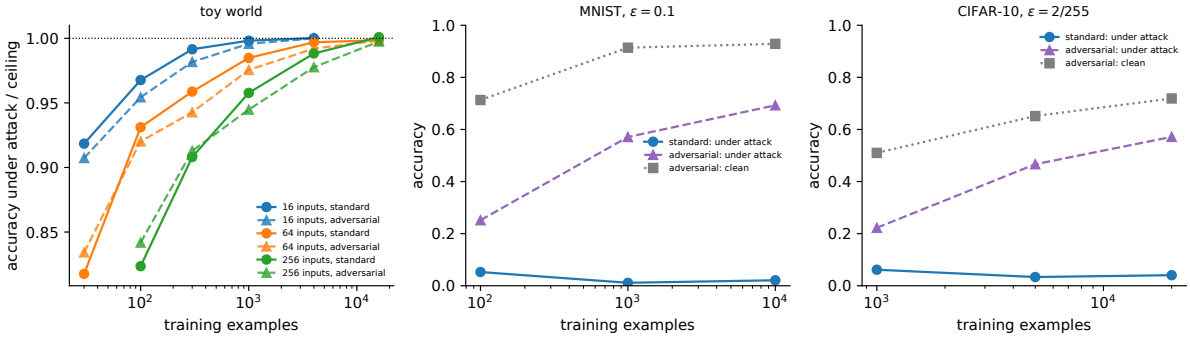


Figure 12: Accuracy under attack against the number of training examples, with standard and adversarial training. Left: the toy world with 16, 64 and 256 inputs, as a fraction of the ceiling (dotted line). Middle and right: MNIST and CIFAR-10; the dotted gray line is the clean accuracy of the adversarially trained network.

Verdict. Under standard training, more data does not change vulnerability, in the toy world or on real data. Under adversarial training, data quantity matters a lot for real data (accuracy under attack rises steeply with n), as Schmidt et al. argue; in the toy world adversarial training needs no more data than standard training to reach the same ceiling, though the data needed grows roughly in proportion to the number of inputs. So data quantity is a cause of *robust learning being hard*, not of standard models being fragile.

3.8 H5: directions the data never use

Claim (Gilmer et al. [9]; Stutz et al. [10]): networks are sensitive in directions off the data manifold, where training gives no signal. **Experiment:** data lie on a 4-dimensional subspace of a 64-dimensional space (plus small off-subspace noise). Attack only along the subspace, or only across it (Figure 13).

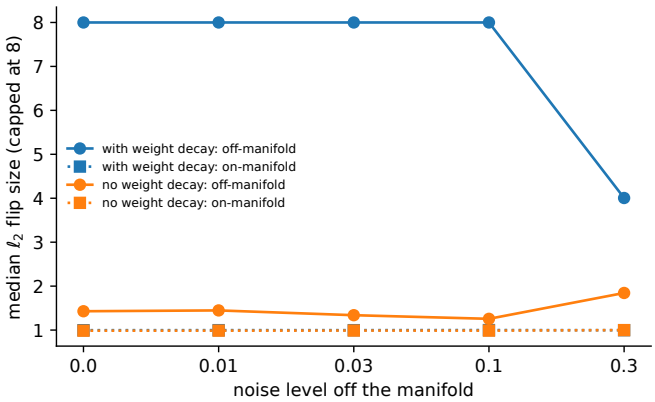


Figure 13: Median total flip size when the attack is limited to the data subspace (squares) or to the directions across it (circles), with and without weight decay. Values above 8 are shown as 8 (no flip found).

With weight decay the network ignores off-manifold directions entirely (no flip found within 8). Without weight decay, off-manifold flip sizes are 1.3–1.8, close to the on-manifold 1.0, and an attack free to use both is about 20% stronger than one limited to the subspace.

Verdict. Real but modest, and *avoidable*: the best rule is blind to these directions and ordinary weight decay makes the network blind too. Contributes; not the main cause here.

3.9 H3: shape versus texture

Claim (Geirhos et al. [5]; for frequency, Yin et al. [6]): networks lean on texture (fine, high-frequency patterns) more than people do, and texture reliance makes them fragile. **World:** 16×16 images with Gaussian pixel noise. A cue is a fixed pattern added with a class-dependent sign; its strength, and hence the accuracy it supports ($\approx 95\%$), is set exactly (Figure 14).

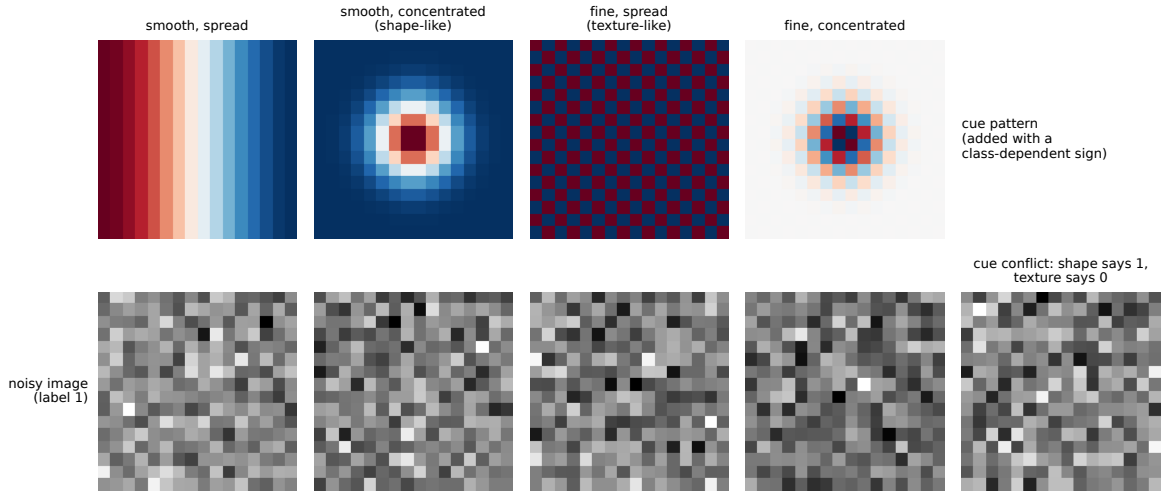


Figure 14: The cues. Top: the four patterns (red positive, blue negative); all four are used in experiment 1, the second and third in experiment 2. Bottom: the same patterns at the strength used, buried in pixel noise, and a cue-conflict image in which the shape cue says class 1 and the texture cue says class 0.

Experiment 1: frequency or spread? Four cues of equal strength: *smooth* or *fine* (a checkerboard), each *spread* over the whole image or *concentrated* in the middle. We report the cue's spread as the ℓ_1/ℓ_2 ratio of its pattern (14.4, 7.1, 16.0, 7.1); Figure 15 (left) shows the results.

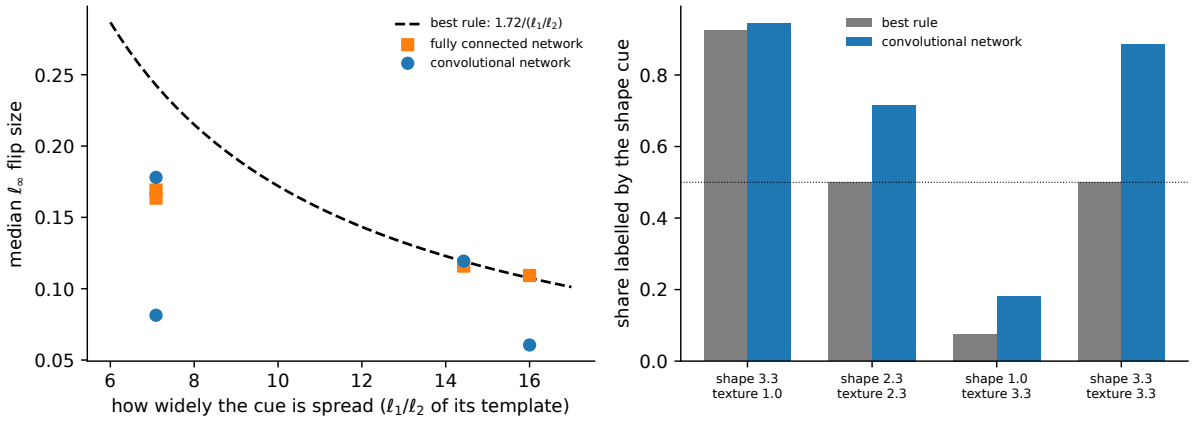


Figure 15: Left: per-input flip size of the best rule (dashed), a fully connected network (squares) and a convolutional network (circles) for four cues of equal strength, against how widely the cue is spread. Right (experiment 2): in images where the shape cue and the texture cue disagree, the share labelled by the shape cue, for the best rule and the convolutional network.

For the best rule, flip size times spread is the same for all four cues (1.70–1.73, equal to the total flip size): *frequency does not matter; spread does.* The fully connected network follows the best rule on the spread cues (0.116 against 0.120; 0.109 against 0.107) and is 1.45× more fragile on the concentrated ones (0.163 against 0.240; 0.169 against 0.242), the same effect as in H4a. The convolutional network (two convolution–ReLU–max-pool stages) matches the best rule on the

smooth cues but not on the fine ones: accuracy falls to 0.87 and 0.86 and the flip size to 0.06 and 0.08 (best rule 0.11 and 0.24), and even the total flip size drops (0.72 against 1.7).

Experiment 2: cue conflict. Images carry both a shape-like cue (smooth, concentrated) and a texture-like cue (fine, spread). In test images the two disagree. The best rule follows whichever cue is stronger, and is indifferent at equal strength. The convolutional network follows the stronger cue too, but at equal strength it is *shape-biased*: it follows the shape cue in 72% and 89% of images for equal strengths 2.3 and 3.3. Flip sizes follow what the model relies on: 0.19 when the shape cue dominates (strengths 3.3 and 1.0) and 0.067 when the texture cue does (1.0 and 3.3), 2.8× more fragile (the spread ratio is 2.3). A worst-case change puts its energy along the cue the model uses (fraction along the shape template 0.39 against 0.03 along the texture template when shape dominates; 0.10 against 0.33 when texture dominates).

Verdict. "Texture" as a category is not a separate cause. For the best rule only spread matters, so a fine pattern is fragile when it is spread over many pixels and not otherwise. Trained networks add architecture-specific effects: this convolutional network reads fine patterns badly (avoidable) and prefers shape at equal strength. Caveat: a 16×16 toy and one architecture; Section 4 checks frequency on real images.

3.10 H9: architecture and invariance

Question: does building in the right invariance reduce vulnerability, or only move it? **World:** 16×16 noisy images; the label is whether a small known pattern (3×3 block) is present *anywhere* (Figure 17). We compare a fully connected network, a small translation-invariant convolutional network (convolution plus a global maximum), and the hand-built matched filter, which knows the pattern (results in Figure 16).

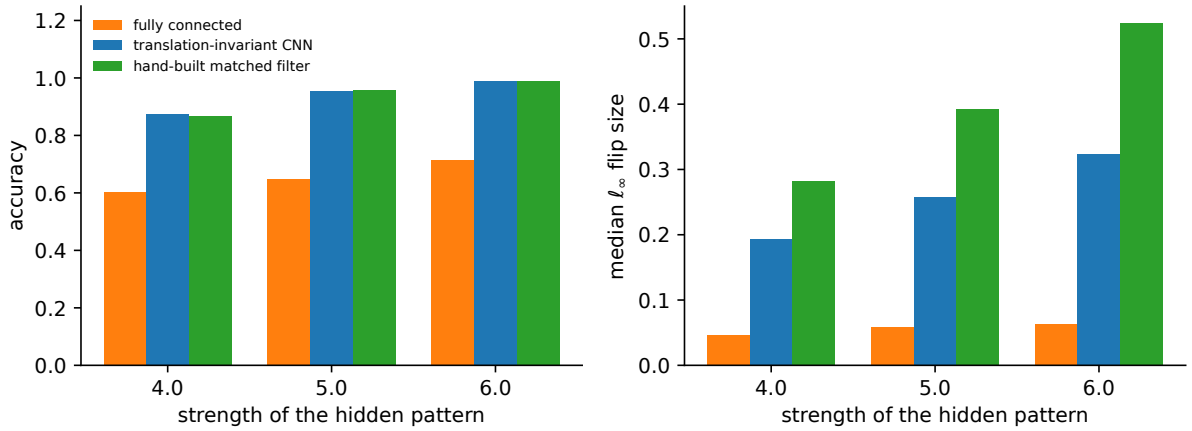


Figure 16: Accuracy (left) and per-input flip size (right) against the strength of the hidden pattern, for three models (3 runs).

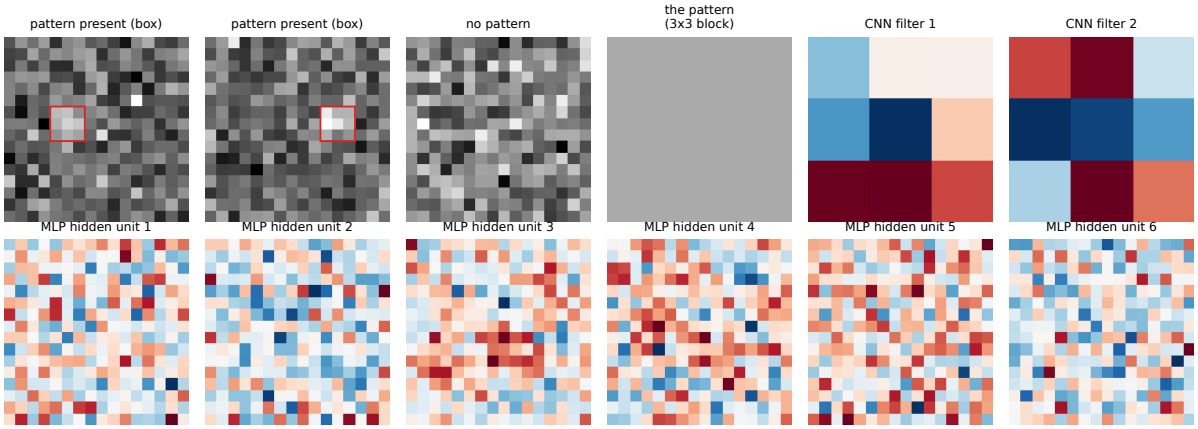


Figure 17: The invariance task and what is learned. Top: images with and without the hidden pattern (red box: where the matched filter finds it), the pattern, and two filters learned by the convolutional network (they resemble the pattern). Bottom: the input weights of six hidden units of the fully connected network, which show no pattern: it has to learn each position separately.

Figure 16 shows the result. The right invariance helps on both axes: at strength 6 the convolutional network reaches accuracy 0.99 with flip size 0.32, while the fully connected network (data-starved with 3000 examples) reaches 0.72 with 0.064. But the learned convolutional network is still about 1.6× more fragile than the matched filter at the same accuracy (0.32 against 0.52 at strength 6; 0.19 against 0.28 at strength 4). We have not diagnosed why.

Verdict. The right inductive bias reduces vulnerability substantially, but a learned solution does not reach the best hand-built one at equal accuracy: an avoidable gap remains. Caveat: the fully connected network is not at equal accuracy, so its flip size mixes data hunger with fragility.

4 Do the toy findings survive on real images?

Each toy hypothesis that can be checked without adversarial training gets a real-data counterpart. Data: MNIST and Fashion-MNIST (small fully connected network) and CIFAR-10 (small convolutional network), 10 000 training images, standard training, flip sizes over 300 test images, 2–3 runs (settings in Appendix A). The ceiling experiment (H10) needs the unknown best robust classifier and is not repeated. Figures 18 and 20 and Table 1 summarise the results.

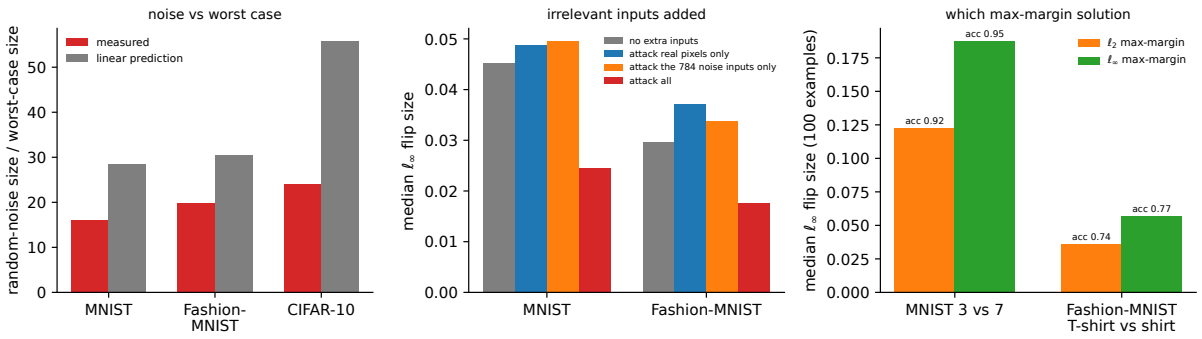


Figure 18: Real-data counterparts of H8, H4a and H7. Left: how many times larger random noise must be than the worst case, measured and as predicted by the linear picture. Middle: flip size after appending 784 irrelevant noise inputs to each image. Right: ℓ_∞ flip size and accuracy of the two max-margin solutions trained on 100 examples.

H8, noise versus the worst case. Random noise must be 16× (MNIST), 20× (Fashion-MNIST) and 24× (CIFAR-10) larger than the worst case to flip a quarter of the points. The linear prediction (ℓ_2 flip size over ℓ_∞ flip size over 0.67) gives 29, 31 and 56: the right order of magnitude, overestimating by 1.3–2.3×. On CIFAR-10 random noise of size 0.076 is needed against 0.003 for the worst case.

H4a, irrelevant inputs. We append 784 pure-noise inputs to every image. Attacking only the real pixels flips at 0.049 (without extra inputs: 0.045), attacking only the noise inputs flips at 0.050, and attacking everything at 0.025. The network is as sensitive to inputs that carry no information as to real ones, and the combined flip size is 1.8× smaller (Fashion-MNIST: 0.037, 0.034, 0.018). This reproduces the toy factor of about two. Figure 19 shows why: the network's gradient with respect to the 784 irrelevant inputs is as large in total as its gradient with respect to the real pixels.

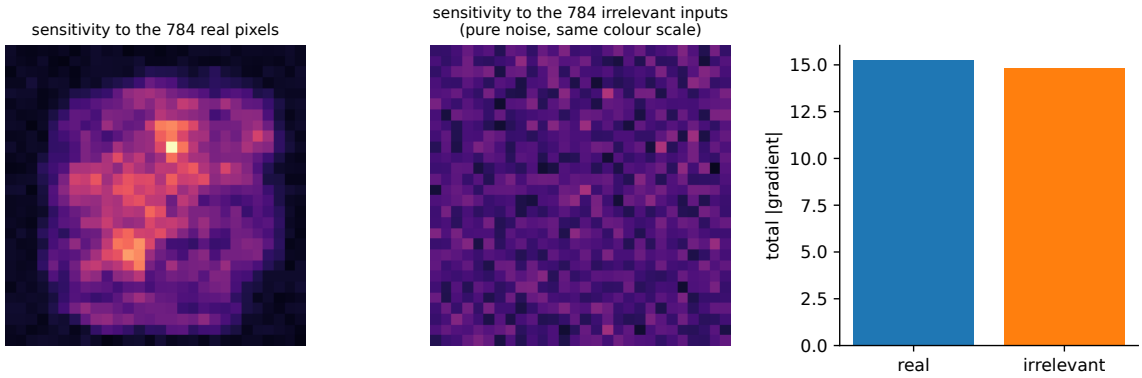


Figure 19: Average size of the loss gradient of the network trained with 784 appended irrelevant inputs, over 500 test images: real pixels (left), irrelevant inputs on the same colour scale (middle), and their totals (right).

H7, which solution. The ℓ_∞ max-margin classifier beats the ℓ_2 one on both accuracy and flip size at every training size we tried (40, 100, 300) on MNIST (3 versus 7) and Fashion-MNIST

(T-shirt versus shirt). With 100 examples: MNIST accuracy 0.954 against 0.919 and flip size 0.188 against 0.122 (+54%); Fashion-MNIST 0.771 against 0.735 and 0.057 against 0.036 (+58%). (The SVM's accuracy on MNIST dips at 300 examples, 0.886, which may be solver tolerance.)

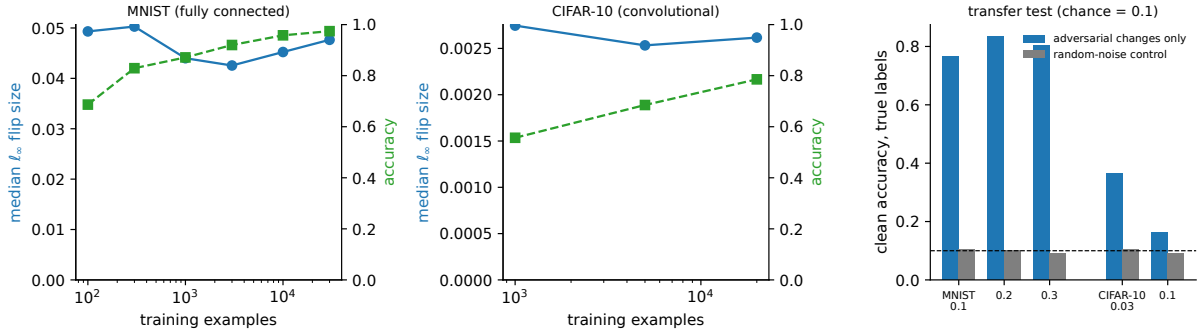


Figure 20: Left and middle (H6): flip size (blue) and accuracy (green) against the number of training examples, for MNIST and CIFAR-10. Right (H2): clean accuracy of a fresh network trained only on adversarially changed images with random target labels (blue) against a random-noise control, for the sizes of change shown (chance = 0.1).

H2, transfer. A fresh network trained only on adversarially changed images with random target labels reaches clean accuracy 0.77, 0.83 and 0.80 on MNIST for changes of size 0.1, 0.2 and 0.3, and 0.37 on CIFAR-10 at 0.03 (0.16 at 0.1); the random-noise controls stay at chance (≈ 0.10). Adversarial changes carry learnable features on real data too, much more weakly on CIFAR-10 with these small networks. Figure 21 shows how the training data for this test look.

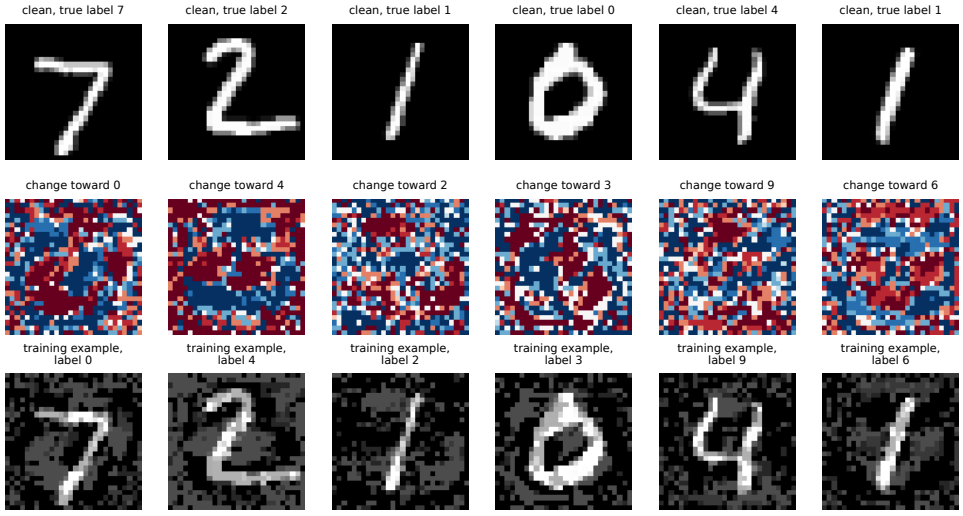


Figure 21: Building the transfer test on MNIST. Top: a clean image and its true label. Middle: the targeted change ($\epsilon = 0.3$) towards a random other label. Bottom: the training example, an image that looks like the original digit but carries the target label.

H6, more data. On MNIST the flip size is flat (0.043–0.050, Figure 20) from 100 to 30 000 examples while accuracy rises from 0.69 to 0.97. On CIFAR-10 it is flat too (0.0027, 0.0025, 0.0026

for 1000, 5000, 20 000 examples; accuracy 0.56, 0.69, 0.79) once every training-set size gets the same number of optimiser steps. (A first run with a fixed number of epochs showed a flip size falling with data; that was an artefact of undertraining the small sets.)

Table 1: Real-data checks of H5, H3 and H9. Median flip sizes (ℓ_2 for the subspace and band rows). PC = principal component.

Check	Data	Condition A	Condition B	Unrestricted	Accuracy
H5: attack on / off the top PCs	MNIST (50 PCs)	on: 1.08	off: 1.42	0.87	0.96
	CIFAR-10 (200 PCs)	on: 0.47	off: 0.12	0.11	0.73
H3: lowest / highest 25% of frequencies	MNIST	low: 0.97	high: 4.41	0.87	0.96
	CIFAR-10	low: 0.27	high: 0.32	0.12	0.73
H9: fully connected / convolutional (ℓ_∞)	MNIST	0.045	0.088	–	0.96 / 0.98
	Fashion-MNIST	0.030	0.029	–	0.85 / 0.87
	CIFAR-10	0.0052	0.0032	–	0.46 / 0.73

H5, off-manifold. On MNIST the network is *less* sensitive off the top 50 components (flip size 1.42) than along them (1.08), with or without weight decay. On CIFAR-10 it is four times more sensitive off the top 200 components (0.12 against 0.47), and an unrestricted attack is no stronger than the off-subspace one (0.11). Caveat: the off-subspace has about 14× more directions than the subspace. Figure 22 shows the smallest change that flips one image, split into its part along the top components and the rest: on MNIST 72% of its energy lies along the top 50 components, on CIFAR-10 only 7% lies along the top 200.

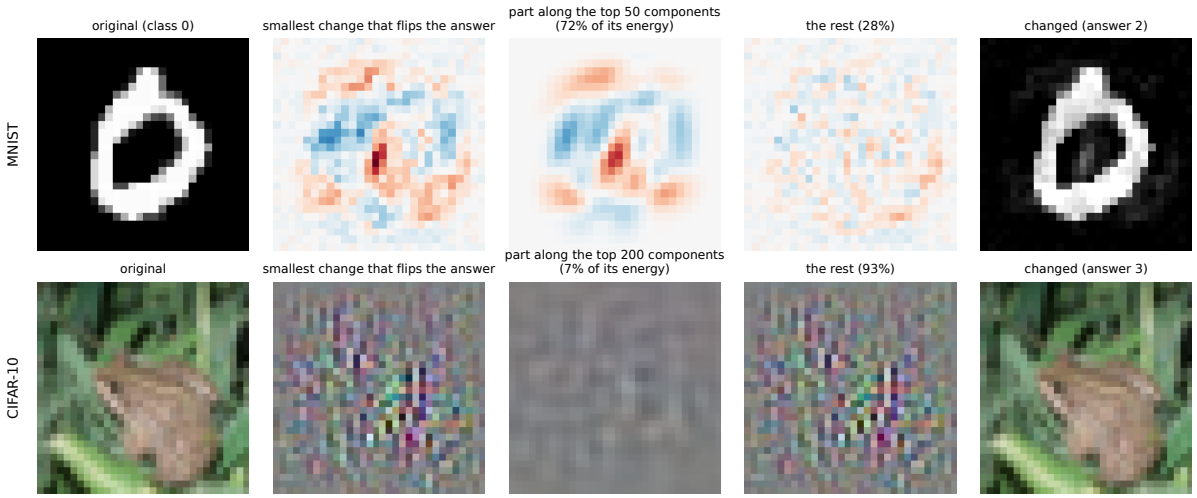


Figure 22: Where the smallest flipping change goes. Columns: the original image, the smallest total (ℓ_2) change that flips the answer (scaled for visibility), its part along the top principal components, the remainder, and the changed image with the new answer.

H3, frequency. Restricting the change to the lowest or highest quarter of the frequencies, the network shows no special fragility to high frequencies: on MNIST it is far *less* sensitive to them (4.4 against 1.0), on CIFAR-10 about equally sensitive (0.32 against 0.27).

H9, architecture. The convolutional network doubles the ℓ_∞ flip size on MNIST (0.088 against 0.045) but is no better on Fashion-MNIST, and on CIFAR-10 it is more accurate (0.73 against 0.46)

but has a *smaller* flip size (0.0032 against 0.0052).

Larger networks. To check that the small networks are not the reason, we repeat the main checks with a 1024-unit, four-layer fully connected network on MNIST and a ResNet-18 on CIFAR-10 (Table 2).

Table 2: Larger networks. Median flip sizes (ℓ_∞ unless marked ℓ_2); same conditions as the small networks.

Check	Wide MLP, MNIST	ResNet-18, CIFAR-10
Accuracy (10 000 / 20 000 images)	0.962	0.809
ℓ_∞ flip size (small network)	0.044 (0.045)	0.0039 (0.0032)
H6: flip size at $n = 100$ / 1000 / 10 000 / 60 000 (MNIST), $n = 1000$ / 5000 / 20 000 (CIFAR-10)	0.038, 0.038, 0.044, 0.052	0.0027, 0.0046, 0.0039
H4a: attack real / noise inputs / all (flip size without noise inputs)	0.045 / 0.048 / 0.024 (0.044)	–
H5: ℓ_2 flip size on / off the top PCs (unrestricted)	1.11 / 1.38 (0.87)	0.47 / 0.16 (0.16)
H3: ℓ_2 flip size, lowest / highest 25% of frequencies	0.98 / 3.87	0.31 / 0.60

The picture does not change. Irrelevant inputs halve the flip size again (1.8×). Flip size is flat in the amount of data on MNIST and varies by less than 1.7× without trend on CIFAR-10. Off-manifold sensitivity appears on CIFAR-10 (3×) and not on MNIST. Neither network is more fragile at high frequencies (the ResNet is 1.9× *less* sensitive to them). The ResNet-18 has a slightly larger flip size than the small convolutional network at much higher accuracy (0.81 against 0.73).

Verdict. Real data mostly confirm the toy mechanisms and the avoidable effects: noise versus worst case (same order), irrelevant inputs (about 1.8×), optimiser bias (+50%), and transfer. They qualify others: more data does not change standard flip sizes, off-manifold sensitivity is a CIFAR-10 effect, high-frequency fragility is not found, and a better architecture is not consistently more robust. Larger networks give the same picture. Caveats: few runs, one dataset per kind, standard training in these checks (adversarial training in Section 3.7).

Two earlier checks complete the picture. Mixing the pixels of MNIST digits with a random rotation (same information, same total distances) leaves total flip size unchanged but shrinks per-input flip size by only 12–14%, not the eightfold drop of the toy world: pixel data are already in the spread-out regime. On CIFAR-10, letting the network use more principal components raises accuracy (44% → 78%) and shrinks *both* flip sizes by about 7–8×, so that loss is a shrinking margin, not spreading. A pilot comparing the networks with nearest-neighbour classifiers shows why "a robust classifier exists" is hard to establish in ℓ_∞ (compare the separation argument of Yang et al. [13]): on raw pixels every pair of images is almost equally far apart (nearest same-class and other-class distances are both about 0.99 on MNIST).

5 Scorecard

Figure 23 gives the scorecard at a glance and Table 3 the evidence behind it.

hypothesis	kind	toy world	real data	note
H1 evidence spread over many inputs	inevitable	supported	partly	geometry holds; MNIST already spread
H2 small, not unreliable; features carry signal	inevitable	supported	supported	transfer works on MNIST, weaker CIFAR-10
H4a irrelevant inputs ignored badly	avoidable	supported	supported	about 2x in toy, MNIST, wide MLP
H4b weight sparsity as a fix	avoidable	supported	not tested	
H4c activity sparsity	none	not supported	not tested	
H8 noise vs worst case	inevitable	supported	partly	right order of magnitude
H10 objective (adversarial training)	avoidable	supported	supported	closes the toy gap; costs accuracy
H7 optimiser bias (l2 vs l_inf margin)	avoidable	supported	supported	+50% flip size on MNIST, F-MNIST
H6 data quantity	none/avoidable	not supported	not supported	none (standard); strong (adversarial)
H5 off-manifold directions	avoidable	partly	partly	CIFAR-10 only, with a caveat
H3 shape vs texture / frequency	inevitable + arch.	partly	not supported	spread, not frequency, matters
H9 architecture / invariance	avoidable	supported	partly	helps MNIST, not consistently

Figure 23: The scorecard at a glance. Green: supported; yellow: partly; red: not supported; gray: not tested. "Kind" is whether the effect is inevitable or avoidable.

Table 3: Scorecard. "Role": necessary (N), sufficient (S), contributes (C), not supported (–). "Kind": inevitable (I) or avoidable (A).

Hypothesis	Role	Kind	Toy world	Real data
H1 evidence spread over many inputs	N, S	I	flip size $\propto 1/\sqrt{k}$ exactly; absent under ℓ_2	geometry holds; MNIST already spread out; CIFAR-10 loss is margin
H2 small, not unreliable	N, S	I	flip size $\propto$ size; adversarial changes teach a fresh network the true classes (10/10)	transfer works: MNIST 0.77–0.83, CIFAR-10 0.37 (chance 0.10)
H4a dense signal / irrelevant inputs	S / C	I / A	dense: same as H1; network up to 2× worse than best rule with irrelevant inputs	appended noise inputs: 1.8× smaller flip size (MNIST), 1.7× (Fashion-MNIST)
H4b weight sparsity	fix	A	removes fragility at an accuracy cost	not tested
H4c activity sparsity	–	–	no effect; fails all at once	not tested
H8 noise sensitivity	(H1)	I	restates H1: ratio $\propto \sqrt{k}$	ratio 16–24, linear prediction 29–56
H10 objective	C	A (new objective)	ceiling 0.75 vs standard 0.45 at $\epsilon = 0.3$; adversarial training reaches 0.74	adversarial training: flip size ×1.5–6, clean accuracy up to −6 points (CIFAR-10), $\approx$ 0 (MNIST)
H7 optimiser bias	C	A	ℓ_∞ max-margin better when signal sparse, data scarce; gradient descent matches SVM	ℓ_∞ solution: +54% (MNIST), +58% (Fashion-MNIST) flip size, higher accuracy
H6 data quantity	– (standard); C (adversarial)	A	no effect under standard training; adversarial training needs no more data than standard, but data needed grows roughly with the number of inputs	flat for standard training (small and large networks); adversarially trained accuracy under attack rises 0.25 → 0.69 with data
H5 off-manifold	C (small)	A	vanishes with weight decay; ∼ 20% without	CIFAR-10: 3–4× more sensitive off top PCs (small CNN, ResNet-18); MNIST: not
H3 shape vs texture	C	I, A	spread is inevitable (best rule: flip size × spread constant, frequency irrelevant); architecture is avoidable (CNN weak on fine patterns, shape-biased)	no special high-frequency fragility (MNIST less, CIFAR-10 equal)
H9 architecture	C	A	invariance: 4–5× flip size over fully connected, 1.6× below hand-built filter	MNIST 2× better; Fashion-MNIST equal; CIFAR-10 smaller flip size

6 Discussion

The first-order cause. In the toy worlds, fragility to small per-input changes is mainly inevitable. The evidence a good classifier uses is spread over many inputs, or carried by clues that are small compared with the allowed change, and a per-input budget lets the attacker push all of it together. Several explanations that look different (noise sensitivity, dense signals, texture reliance as a category) turn out to be the same mechanism seen from other sides: for the best rule only how widely the evidence is spread matters.

Avoidable causes. They exist and can be sizeable: up to a factor of two from ignoring irrelevant inputs, 1.6× from a learned rather than a designed filter, a convolutional network that reads fine patterns badly, and a large gap whenever the training objective ignores robustness. But they sit on top of the inevitable part and cannot remove it, and adversarial training removes the avoidable gap only at a price in clean accuracy.

How far this goes. The real-data checks, with small and large networks, support the mechanism and several avoidable effects (irrelevant inputs, optimiser bias, transfer, the objective) but not every toy prediction (off-manifold sensitivity, architecture). So this remains a hypothesis about large networks, not a result.

What would change our mind. A real task where the classes are well separated in ℓ_∞, a robust classifier is easy to write down, and standard training still finds a fragile one, would be a case where the avoidable part dominates. Finding or constructing such a case on real data is the most valuable open test.

7 Limits

The study has several limits. The toy worlds use independent Gaussian clues and small networks. Attacks are one-step or projected gradient attacks. Adversarial training is studied only in the data-quantity and objective experiments, and real-data checks use few runs. Weight decay is on by default and affects H5 and H6. Finally, "root cause" is strong language: we claim a map of causes and their rough size in controlled worlds, not a single answer for large networks.

A Settings and code map

Toy task: independent Gaussian clues, mean $(y - \frac{1}{2})\Delta\mu_i$, standard deviation σ_i, unclipped; reliable clue $\Delta\mu = 1.0$, $\sigma = 0.3$; weak clues $\Delta\mu = 0.4$, $\sigma = 0.25$. Networks: one hidden layer of 8 units (32 in H5), Adam with learning rate 0.02, 500 full-batch steps, weight decay 10^{-2} unless stated, 4000 training and 2000 test points, 10 runs. Flip size: exact for linear rules, per-point bisection over a 20-step attack for networks. Code: `study/toy_experiments.py` (H4a, H5–H8, H10), `study/shape_texture.py` (H3), `study/invariance.py` (H9), the H2 transfer test (`run_h2b`) and the gradient-descent check for H7 (`run_h7gd`) in `study/toy_experiments.py`, `study/pilot_gap.py`, figures from `study/paper_figures.py`; adversarial training in `study/adv_training.py`, larger networks in `study/real_big.py`, H1 and H4b–c use the earlier scripts in the repository root, and the real-data checks are in `study/real_checks.py` (functions `r2b`, `r3`, `r4a`, `r5`–`r9`).

Adversarial training and larger networks. Adversarial training: ℓ_∞ PGD with 5 steps (step $2.5\epsilon/5$, random start) on every batch; evaluation by a 20-step PGD attack; $\epsilon = 0.3, 0.2, 0.1$ in the toy world, 0.1 on MNIST and 2/255 on CIFAR-10; training sizes get a matched number of optimiser steps (≥ 1500). Larger networks: a four-layer MLP with 1024 units per layer (MNIST) and a ResNet-18 (CIFAR-10), SGD with momentum and a one-cycle schedule, matched step budgets, 2–3 runs.

References

[1] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus. *Intriguing properties of neural networks*. ICLR, 2014.

[2] I. Goodfellow, J. Shlens, C. Szegedy. *Explaining and Harnessing Adversarial Examples*. ICLR, 2015.

[3] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, A. Madry. *Robustness May Be at Odds with Accuracy*. ICLR, 2019.

[4] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, A. Madry. *Adversarial Examples Are Not Bugs, They Are Features*. NeurIPS, 2019.

[5] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, W. Brendel. *ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness*. ICLR, 2019.

[6] D. Yin, R. G. Lopes, J. Shlens, E. D. Cubuk, J. Gilmer. *A Fourier Perspective on Model Robustness in Computer Vision*. NeurIPS, 2019.

[7] N. Ford, J. Gilmer, N. Carlini, E. D. Cubuk. *Adversarial Examples Are a Natural Consequence of Test Error in Noise*. ICML, 2019.

[8] L. Schmidt, S. Santurkar, D. Tsipras, K. Talwar, A. Madry. *Adversarially Robust Generalization Requires More Data*. NeurIPS, 2018.

[9] J. Gilmer, L. Metz, F. Faghri, S. S. Schoenholz, M. Raghu, M. Wattenberg, I. Goodfellow. *Adversarial Spheres*. ICLR Workshop, 2018.

[10] D. Stutz, M. Hein, B. Schiele. *Disentangling Adversarial Robustness and Generalization*. CVPR, 2019.

[11] D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar, N. Srebro. *The Implicit Bias of Gradient Descent on Separable Data*. JMLR, 2018.

[12] C.-J. Simon-Gabriel, Y. Ollivier, L. Bottou, B. Schölkopf, D. Lopez-Paz. *First-order Adversarial Vulnerability of Neural Networks and Input Dimension*. ICML, 2019.

[13] Y.-Y. Yang, C. Rashtchian, H. Zhang, R. Salakhutdinov, K. Chaudhuri. *A Closer Look at Accuracy vs. Robustness*. NeurIPS, 2020.